

1 Proof Techniques

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-10-12 16:06

Learning Outcomes

Unit 1: ***Proof Techniques***

1. Know logical *proof strategies* for proving implications, set inclusions, set equalities, and quantified statements.
2. Be able to use *mathematical induction* in simple proofs.
3. Know techniques for *proving termination* and *correctness* of procedures.

Outline

1 Proof Techniques

1.1 Digression: Random Shuffle

1.2 Proof Templates

1.3 Mathematical Induction

1.4 Correctness Proofs

1.1 Digression: Random Shuffle

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.

- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right?*

Clicker Question



Select all statements that apply to myShuffle (for you).

- **A** I have seen this shuffling algorithm (or a very similar method) before.
- **B** I can understand the pseudocode for myShuffle (so I would be able to do an example by hand).
- **C** It can generate all possible orderings of A (depending on the random numbers).
- **D** myShuffle produces all possible orderings with the same probability.
- **E** Assuming randomInt gives (perfect) uniform random numbers in the given range, myShuffle generates any ordering with equal probability.



→ *sli.do/effalg*

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right?*



$n = 2$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1  procedure myShuffle(A[0..n))
2      for i := 0, ..., n − 1
3          r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4          Swap A[i] and A[r] // Swap A[i] to random position.
5      end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right??*



$n = 2$ $n = 3$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right???*



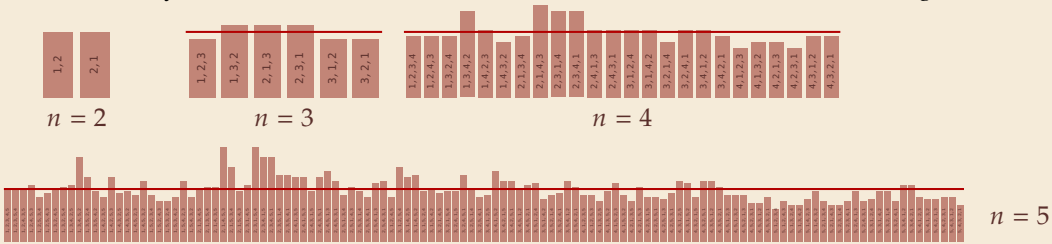
$n = 2$ $n = 3$ $n = 4$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right????*



Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

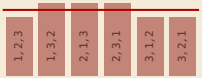
```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

← **WRONG! DO NOT USE**

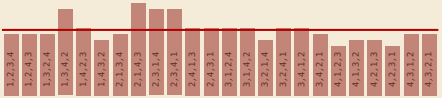
- Intuitively: *All elements are moved to a random index, so the order is random . . . right????*



$n = 2$



$n = 3$



$n = 4$



$n = 5$

Clicker Question

Select all statements that apply to myShuffle (for you).



A I have seen this shuffling algorithm (or a very similar method) before. ✓

B I can understand the pseudocode for myShuffle (so I would be do an example by hand). ✓

C It can generate all possible orderings of A (depending on the random numbers). ✓

D ~~myShuffle produces all possible orderings with the same probability.~~

E ~~Assuming randomInt gives (perfect) uniform random numbers in the given range, myShuffle generates any ordering with equal probability.~~

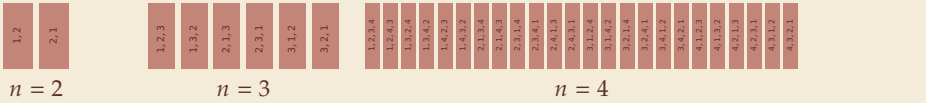


→ *sli.do/effalg*

Correct shuffle

- interestingly, a very small change corrects the issue

```
1  procedure shuffleKnuthFisherYates(A[0..n))
2      for i := 0, ..., n − 1
3          r := randomInt([i..n))
4          Swap A[i] and A[r]
5      end for
```



- looks good . . .
- . . . but how can we convince ourselves that it is correct, *beyond any doubt?*

1.2 Proof Templates

What is a *formal* proof?

A formal proof (in a logical system) is a **sequence of statements** such that each statement

1. is an *axiom* (of the logical system), or
2. follows from previous statements using the *inference rules* (of the logical system).

Among experts: Suffices to *convince a human* that a formal proof *exists*.

But: Use formal logic as guidance against faulty reasoning. $\rightsquigarrow$ bulletproof



What is a *formal* proof?

A formal proof (in a logical system) is a **sequence of statements** such that each statement

1. is an *axiom* (of the logical system), or
2. follows from previous statements using the *inference rules* (of the logical system).

Among experts: Suffices to *convince a human* that a formal proof *exists*.

But: Use formal logic as guidance against faulty reasoning. $\rightsquigarrow$ bulletproof



Notation:

- Statements: $A \equiv$ "it rains", $B \equiv$ "the street is wet".
- Negation: $\neg A$ "Not A"
- And/Or: $A \wedge B$ "A and B"; $A \vee B$ "A or B or both" (non-exclusive or)
- Implication: $A \Rightarrow B$ "If A, then B"; $\neg A \vee B$
- Equivalence: $A \Leftrightarrow B$ "A holds true *if and only if ('iff')* B holds true."; $(A \Rightarrow B) \wedge (B \Rightarrow A)$

Clicker Question



Is the following statement true?

"If the Earth is flat, then ships can fall over its rim."

$\neg A \vee B$

A

B

A Yes **B** No **C** Neither



→ *sli.do/effalg*

Clicker Question



Is the following statement true?
"If the Earth is flat, then ships can fall over its rim."

A Yes ✓ **B** ~~No~~ **C** ~~Neither~~



→ *sli.do/effalg*

Implications

$A \Rightarrow B \quad \equiv \quad \neg A \vee B$

To prove $A \Rightarrow B$, we can

$\neg(\neg B) \vee (\neg A) \quad \equiv \quad \neg A \vee B$

- directly derive B from A *direct proof*
- prove $(\neg B) \Rightarrow (\neg A)$ *indirect proof, proof by contraposition* (Kontraposition)
- assume $A \wedge \neg B$ and derive a contradiction *proof by contradiction, reductio ad absurdum* (Widerspruchsbeweis)
- distinguish cases, i. e., separately prove $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$. *proof by exhaustive case distinction*

Clicker Question

$n = 2 \cdot k + 1$

$k \in \mathbb{N}_0$

$n^2 = (2k+1)^2$

$= 4k^2 + 4k + 1$

$= 2k' + 1$

$k' = 2k^2 + 2k$

$\in \mathbb{N}_0$

Suppose we want to prove:
"If $n^2 \in \mathbb{N}_0$ is an even number, then n is also even."
For that we show that when n is odd, also n^2 is odd.
Which proof template do we follow?



A direct proof: $A \Rightarrow B$

B indirect proof: $(\neg B) \Rightarrow (\neg A)$

C proof by contradiction: $A \wedge \neg B \Rightarrow \lightning$

D proof by case distinction: $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$



→ *sli.do/effalg*

Clicker Question



Suppose we want to prove:
"If $n^2 \in \mathbb{N}_0$ is an even number, then n is also even."
For that we show that when n is odd, also n^2 is odd.
Which proof template do we follow?

A ~~direct proof: $A \Rightarrow B$~~

B indirect proof: $(\neg B) \Rightarrow (\neg A)$ ✓

C ~~proof by contradiction: $A \wedge \neg B \Rightarrow \lightning$~~

D ~~proof by case distinction: $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$~~



→ sli.do/effalg

Equivalences

To prove $A \Leftrightarrow B$,
we prove both implications $A \Rightarrow B$ and $B \Rightarrow A$ separately.

(Often, one direction is much easier than the other.)

queue

Set Inclusion and Equality

To prove that a set S contains a set R, i. e., $R \subseteq S$,
we prove the implication $x \in R \Rightarrow x \in S$.

To prove that two sets S and R are equal, $S = R$,
we prove both inclusions, $S \subseteq R$ and $R \subseteq S$ separately.